



USENIX

THE ADVANCED COMPUTING
SYSTEMS ASSOCIATION

Bayesian Code Diffusion for Efficient Automatic Deep Learning Program Optimization

Isu Jeong and Seulki Lee, *Ulsan National Institute of Science and Technology*

<https://www.usenix.org/conference/osdi25/presentation/jeong>

This paper is included in the Proceedings of the 19th USENIX Symposium
on Operating Systems Design and Implementation.

July 7–9, 2025 • Boston, MA, USA

ISBN 978-1-939133-47-2

Open access to the Proceedings of the 19th USENIX Symposium
on Operating Systems Design and Implementation is sponsored by



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

Bayesian Code Diffusion for Efficient Automatic Deep Learning Program Optimization

Isu Jeong Seulki Lee

Ulsan National Institute of Science and Technology (UNIST)

Abstract

We introduce Bayesian code diffusion, a new deep learning program optimization strategy devised to accelerate the auto-tuning process of deep learning compilers. Using the concepts of *prior* and *posterior* distributions in the Bayesian framework and reformulating them in the context of deep learning program optimization, the proposed approach efficiently searches for optimal program code in a significantly reduced search space through an iterative diffusion of program code. To further enhance the efficiency of program optimization, we propose pre-training and fine-tuning for the cost model, which improves both the model’s predictive accuracy and training efficiency. We implement Bayesian code diffusion in Ansor and evaluate its performance on a wide range of deep learning models on both CPUs and GPUs. Existing approaches struggle to reliably generate high-performing deep learning programs, i.e. achieving low program execution latency, across various configurations, including diverse deep learning model architectures and hardware platforms (CPU and GPU). In contrast, Bayesian code diffusion reduces the end-to-end compilation (optimization) time required to generate the equivalent program execution latency in various configurations, i.e., achieving up to $3.31\times$ optimization speedup. This substantial improvement demonstrates that Bayesian code diffusion performs efficient and principled deep learning program optimization across a wide range of deep learning models, operators, and hardware (CPU and GPU).

1 Introduction

High-performance deep learning (tensor) programs are essential for constructing efficient deep learning systems. To achieve this, deep learning compilers, such as TVM [6], Glow [34], nGraph [10] and XLA [23], have been developed to generate deep learning programs optimized for various hardware architectures, such as CPUs, GPUs, and others. Without relying on hardware vendor-specific libraries, e.g., BLAS [22], Intel MKL [19], Nvidia cuBLAS [29], and

cuDNN [30], they perform program optimizations by exploring the search space constructed from pre-defined rules and patterns, such as templates [6]. The search space is navigated by tuning its associated parameters, minimizing manual effort.

However, these deep learning compilers often result in suboptimal program performance, resulting in increased program execution latency. To generate improved deep learning (tensor) programs, the concept of auto-tuning [7] has been proposed, where a set of parameters that define the program codes are automatically searched and optimized. Notably, Ansor [47] constructs an expanded search space for each subgraph of the deep learning model through sketch generation, followed by random initialization and evolutionary search [41]. Although such auto-tuning techniques can identify deep learning programs that achieve lower execution latencies, extensive program exploration in an expanded search space often leads to increased program optimization time.

The increased optimization time observed in auto-tuning approaches highlights three opportunities for improvement. First, each subgraph is assigned an independent search space with a distinct set of optimizations, despite potential overlaps between them, which may be redundant. These redundancies present opportunities for improvement through reuse or shared optimization across subgraphs. Second, many existing auto-tuning methods [7, 47] perform a large number of random searches to identify optimal program codes, which can be time-consuming and cause inefficiencies in program optimization, despite the opportunity to start from better initial points. Lastly, the cost model used to predict the expected performance of the generated program, i.e., program execution latency, is trained without a learning strategy, presenting an opportunity to improve both predictive accuracy and learning efficiency through more principled training strategies. Although various approaches [12, 14, 26] have been proposed to exploit these opportunities, each has limited effectiveness in fully resolving these issues and presents its own set of limitations, as summarized in Tab. 1.

In this paper, we introduce the concept of Bayesian code diffusion and demonstrate its implementation, which enhances

Table 1: The comparison of state-of-the-art auto-tuning methods for deep learning (tensor) program optimization relative to Ansor [47] shows that the proposed Bayesian code diffusion enhances all features.

	Sample Program Sharing	Search Strategy Enhancement	Cost Model Improvement	Online Learning	Applicable HW	
					CPU	GPU
SelectiveTuning (2019) [3]	✓	✓	✗	✓	✗	✓
Ansor (2020) [47]	✗	✗	✗	✓	✓	✓
ETO (2021) [12]	✓	✓	✗	✓	✗	✓
DietCode (2022) [46]	✓	✗	✗	✓	✗	✓
TransferTuning (2022) [14]	✓	✓	✗	✗	✓	✗
OneShotTuner (2022) [35]	✗	✗	✓	✗	✓	✓
FamilySeer (2023) [26]	✗	✗	✓	✓	✓	✓
BayesianCodeDiffusion (ours)	✓	✓	✓	✓	✓	✓

the efficiency of auto-tuning for optimizing deep learning (tensor) programs. We reformulate the problem of deep learning program optimization based on the Bayesian framework [27] and present an efficient solution for addressing it. Specifically, we leverage the concept of *prior*, represented by well-optimized program code for a specific subgraph, and propagate its parameter to a *posterior* subgraph—another similar subgraph to be optimized—to facilitate an efficient search for optimal programs in the program parameter space. Inspired by the observation that deep learning models often share similar subgraphs in their architectures, we iteratively transform the *posterior* program within the highly-likely parameter spaces close to the *prior*’s parameter distribution through the process we term *code diffusion*. This process enables an efficient and principled optimization of program parameters, in contrast to existing auto-tuning methods [7, 47] applying random search in an expansive program parameter space.

We also propose a cost model learning strategy (i.e., pre-training and fine-tuning) that adjusts the optimization order of program subgraphs to improve the convergence and predictive accuracy of the cost model. Auto-tuning generates a large number of program candidates, but measuring their latency on hardware is costly. To reduce this overhead, a cost model is trained during program compilation (optimization) to estimate the program latency based on a small subset of hardware measurements, following an online learning approach. However, existing approaches, such as round-robin or gradient-based sampling [47], yield overly diverse data samples, hindering the cost model’s ability to generalize. In contrast, the proposed pre-training and fine-tuning strategy guides the cost model through a more structured data sequence, improving both learning efficiency and prediction quality. Initially, the cost model is trained using a set of *prior* program codes across subgraph clusters in a random order—we refer to as *pre-training*—to enhance its generalization capacity by learning diverse data generated through hardware measurements of heterogeneous *prior* subgraphs. Then, the cost model is trained with specific and homogeneous data generated by each cluster of similar subgraphs one by one—we refer to as *fine-tuning*—to improve its predictions on each subgraph cluster, on top of the generalization ability obtained during pre-training.

We implement the proposed Bayesian code diffusion on

Ansor [47], demonstrating that it can be seamlessly and easily integrated into existing auto-tuning frameworks with minimal modification and effort. We conduct extensive experiments across a wide range of deep learning models, such as ResNet [16] and BERT [11]. The experimental results show that Bayesian code diffusion accelerates the end-to-end compilation (optimization) time of deep learning models by up to $3.31\times$ while producing programs with execution latencies up to $1.13\times$ faster than those generated by state-of-the-art auto-tuning methods [47]. The reduced compilation time is achieved by effective program sharing, efficient and principled program search, and online optimization on both CPU and GPU, without compromising on program execution latency. In contrast, previous methods are often limited by inefficient program search capabilities [26, 47], or constrained hardware compatibility (CPU/GPU) [3, 12, 46], or limited support for operators [46]. Some also rely on pre-compiled programs [14] or offline-trained cost models [35].

2 Background and Opportunities

2.1 Deep Learning Compilers and Auto-Tuning

Deep Learning Compilers. To accelerate deep learning workloads on target hardware architectures, specialized hardware vendor libraries, such as BLAS [22], Intel MLK [19], Nvidia cuBLAS [29], and cuDNN [30], have been widely used to generate efficient deep learning programs. However, developing these hardware-dependent libraries requires significant time and resources, making it difficult to keep up with the rapid emergence of state-of-the-art deep learning models and new hardware platforms. Intermediate Representation (IR)-based deep learning compilers [6] try to tackle this problem by being designed to operate across various hardware architectures without being constrained by hardware dependencies, alleviating the burden of manual optimization. These deep learning compilers, such as TVM [6], Glow [34], nGraph [10] and XLA [23], take deep learning workloads as input and apply a range of optimizations at both high-level and low-level IR to generate efficient deep learning program code without relying on hardware vendor libraries.

Auto-Tuning. Auto-tuning [7, 47] is a technique that automates the hardware-specific optimizations of the low-level IR lowered from the high-level IR graph—e.g., Relay IR [33] in TVM and HLO IR [15] in XLA—of the deep learning model. In the backend of the compiler, it sketches for optimal program parameters, such as loop tile size, from a vast set of candidates to ensure that the final program code performs best on the target hardware. For example, to explore efficient code implementation for the given tensor operations, AutoTVM [7] builds the search space of the parameters based on manually defined templates which demand considerable time and effort in their design. Consequently, the search space becomes dependent on manually predefined templates, which restricts the search scope and range. To solve this problem, Ansor [47] automatically constructs the search space through program sampling and fine-tuning stages, finding optimal program code without being restricted by limited templates.

Fig. 1 illustrates the auto-tuning process of Ansor [47]. Ansor first constructs the search space by generating a set of optimization rules, named sketch, for each subgraph. It then selects a subgraph and performs random initialization for the parameters of optimization rules belonging to the sketch, followed by iterative fine-tuning to optimize the parameters of these rules and generate executable complete program code. Among the resulting candidates, only those predicted by the cost model to have low latency are retained and evaluated on the target hardware. The cost model is trained with these evaluation results from scratch in an online learning paradigm. This process is repeated until the time budget is exhausted.

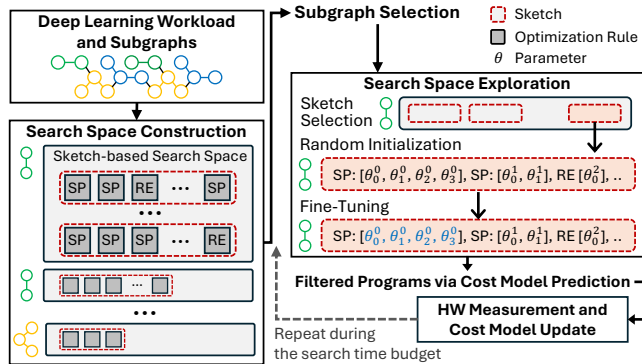


Figure 1: An overview of auto-tuning process in Ansor [47]. The details of the optimization rule, SP (Split-Step), and its parameters θ (integers) are described in Sec. 4.3.

2.2 Opportunities of Auto-Tuning

Although Ansor [47] automates finding optimal hardware-specific program code through program sampling and fine-tuning, it presents several opportunities, as described below.

Opportunity 1: Ignored Subgraph Similarity. In auto-tuning, the computation graph (i.e., high-level IR) is divided into subgraphs that can be optimized via pattern mapping or specific logic. Ansor [47] constructs the search space for

each subgraph by recursively applying predefined optimization rules, such as reordering, multi-level tiling, and node fusion. Since this set of rules, named sketch, lacks specific parameters that define tile sizes and loop annotations, they must be determined by tuning (optimizing) parameters to generate executable programs. However, each parameter is determined independently through random initialization to produce executable program code. As a result, the optimal program parameters identified by one subgraph are not shared with others, even though there is potential for reuse when subgraphs exhibit similarities.

To reuse optimized program code, TransferTuning [14] adapts parameters previously optimized for a different deep learning model and transforms them to fit the recipient subgraph of the target deep learning model. However, this approach requires a pre-compiled deep learning program that closely resembles the recipient deep learning program, which limits its usability and applicability in practice. Although DietCode [46] creates a unified search space for microkernel operations and collectively optimizes them, it supports only a limited set of operators (e.g., dense and batch matmul), requiring the development of each individual operation and restricting implementations to only GPUs.

Opportunity 2: Randomized Fine-Tuning Initialization.

To fine-tune program codes, Ansor [47] begins with a set of candidate complete programs, known as schedules, obtained through a combination of sketch generation and random parameter initialization. These initial programs are then iteratively fine-tuned with search algorithms, such as evolutionary operations [41], to produce new programs that can achieve improved performance, typically lower program execution latency. However, since these initial candidate programs are generated randomly, a large number of fine-tuning iterations is typically required to identify the optimal program code.

To reduce extensive random sampling and fine-tuning, SelectiveTuning [3] selects a subgraph in the high-level IR and optimizes its parameters through sufficient searches. The optimized parameters are then adapted to other subgraphs of similar structure through a one-time transformation based on predefined rules. While this approach can reduce random search iterations, relying on a one-shot single transformation without sufficient fine-tuning often results in suboptimal program performance, thus limiting its overall effectiveness.

Opportunity 3: Learning Strategy of Cost Models. Auto-tuners employ a cost model [47] that predicts the expected performance of the generated programs (i.e., execution latency) to select ones anticipated to perform better than others.

The cost model is trained from scratch during compilation using hardware measurements in an online learning setting. Since optimization depends on the cost model, both the quality and quantity of the training data are critical for effective auto-tuning. In general, the training set is derived from programs executed using the round-robin and gradient with respect to program latency, without considering similarities, resulting in

the cost model being trained on overly broad and diverse data. If so, it must make predictions on a wide range of diverse programs, which may result in suboptimal prediction accuracy. Moreover, the diversity of data can prolong the training time, leading to unreliable predictions until the model convergence. Additionally, if there exist similar programs, they tend to produce similar data samples, causing the model to learn redundant and ineffective samples. This not only fails to enhance the model’s learning performance but also wastes time and resources [26, 32, 42].

To accelerate convergence and improve prediction performance of the cost model, FamilySeer [26] groups similar subgraphs and trains a separate cost model for each group. Although this approach can reduce program search time by using a dedicated cost model for each group, the limited range and diversity of training datasets can often result in suboptimal programs, as each group is likely to generate only similar program samples. Moreover, it does not reuse optimized parameters across subgraphs, resulting in inefficient search.

3 Methodological Idea

To take advantage of the aforementioned opportunities of auto-tuning in deep learning compilers, we propose Bayesian code diffusion, of which the key idea is illustrated in Fig. 2.

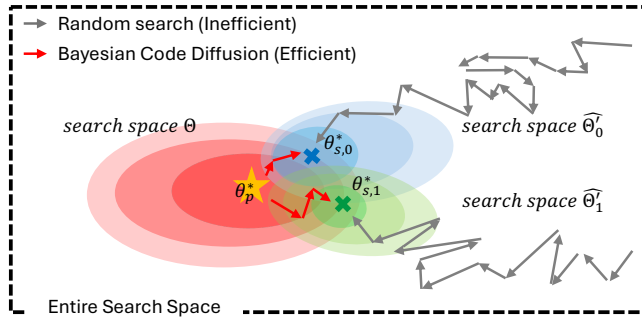


Figure 2: The concept of Bayesian code diffusion: a sufficiently optimized *prior* parameter of one subgraph (θ_p^*) is propagated to similar subgraphs (*posteriors*). Then, *posterior* parameters ($\theta_{s,\{0,1\}}^*$) are derived and refined from the *prior* for each subgraph via code diffusion using a Bayesian formulation, enabling efficient deep learning program optimization.

It first selects a *prior* subgraph and performs a sufficient search to find its optimal program parameters. These optimized parameters are then propagated to structurally similar subgraphs, referred to as *posterior* subgraphs, from a Bayesian perspective. The initial search space of each *posterior* subgraph is restricted to that of the *prior* subgraph, and optimization is performed in the vicinity of the *prior*’s optimal parameters through *prior* propagation. Then these parameters are fine-tuned by adding some noise to find the optimal configuration for the *posterior* subgraph. This process is referred to as *code diffusion*. Repeating this code diffusion process enables the effective propagation of the most promising area

of the search space among similar subgraphs, effectively reducing fine-tuning iterations. Also, in this process, the cost model undergoes pre-training using *prior* subgraphs and is subsequently fine-tuned on *posterior* subgraphs, allowing it to learn efficiently and effectively. The following subsection presents the hypothetical formulation of Bayesian code diffusion (Sec. 3.1) and empirical findings that support the idea and validity (Sec. 3.2).

3.1 Theoretical Formulation

Prior Parameters. Bayesian code diffusion first begins by clustering subgraphs of the deep learning model according to their similarities and selecting one of them as the *prior* subgraph $\mathcal{G}_p$ for each cluster. Given $\mathcal{G}_p$, the optimization task is to find the optimal parameter θ_p^* generating optimal program code for $\mathcal{G}_p$. If we employ the Bayesian formulation to this, with an abuse of notations, the optimization task can be expressed as maximizing the following conditional likelihood.

$$\theta_p^* = \underset{\theta_p}{\operatorname{argmax}} f_{\min}(c(\theta_p) \mid \theta_p) \quad (1)$$

Here, $c(\theta)$ is the program code defined by the parameter θ , and $f_{\min}(c(\theta))$ is defined as the hypothetical probability density regarding whether the execution latency of $c(\theta)$ is the minimal or not among all $\theta \in \Theta$ explored so far, as follows:

$$f_{\min}(c(\theta)) = \begin{cases} 1 & \text{if } l(c(\theta)) < l(c(\theta')) \forall \theta' \in \Theta \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

where $l(c(\theta))$ is the execution latency of $c(\theta)$, and Θ is the total set of schedules explored so far. Now assume a subgraph $\mathcal{G}_s$, presumed to be similar to $\mathcal{G}_p$, is given, and the task is to find its optimal parameter θ_s^* . In principle, the optimal program code $c(\theta_s^*)$, which yields the minimum latency for $\mathcal{G}_s$, can be found if all possible parameters $\theta \in \hat{\Theta}$ are explored. This can be expressed as a hypothetical marginal distribution over all possible $\theta \in \hat{\Theta}$, which is given by:

$$f_{\min}(c(\theta_s^*)) = \sum_{\theta \in \hat{\Theta}} f_{\min}(c(\theta) \mid \theta) f(\theta) \quad (3)$$

In Ansor [47], the search space $\hat{\Theta}$ is defined by a sketch—a sequence of optimization rules—and consists of all possible parameter combinations for those rules, defined as $\hat{\Theta} = \prod_{i=0}^K \theta_i$, where K is the number of rules belonging in the sketch. However, due to the vastness of $\hat{\Theta}$, exhaustively exploring all configurations $\theta \in \hat{\Theta}$ is infeasible, necessitating the derivation of a feasible yet effective subspace $\hat{\Theta}' \subseteq \hat{\Theta}$.

Prior Propagation. To reduce $\hat{\Theta}$ into a feasible yet effective search space $\hat{\Theta}'$, we take θ_p^* , optimized for the *prior* subgraph $\mathcal{G}_p$ in Eq. (1), as the *prior* parameter for $\mathcal{G}_s$ by setting $\hat{\Theta}' = \{\theta_p^*\}$. Specifically, we may regard θ_p^* as the mode of the hypothetical Gaussian distribution for θ_p , i.e.,

$f(\theta_p) = \mathcal{N}(\theta_p^*, \sigma_p^2)$ with the mean θ_p^* and variance σ_p^2 . Based on our observations that θ_p^* and θ_s^* are likely to be close in the search space, i.e., $\theta_p^* \simeq \theta_s^*$ as $\mathcal{G}_p \simeq \mathcal{G}_s$, we also assume the hypothetical distribution of θ_s is also likely to be close to that of θ_p , and thus take $f(\theta_p) = \mathcal{N}(\theta_p^*, \sigma_p^2)$ as the hypothetical *prior* distribution of θ_s as $f(\theta_s) = \mathcal{N}(\theta_p^*, \sigma_p^2)$. From this, we can avoid computing the intractable marginal in Eq. (3) over all possible $\theta \in \hat{\Theta}$, but instead evaluate the following hypothetical *posterior* as a joint distribution:

$$\theta_s^* = \operatorname{argmax}_{\theta_s \in \hat{\Theta}'} f_{\min}(c(\theta_s), \theta_s) = \operatorname{argmax}_{\theta_s \in \hat{\Theta}'} f_{\min}(c(\theta_s) \mid \theta_s) f(\theta_s) \quad (4)$$

where $\hat{\Theta}' = \{\theta_p^*\}$ for the beginning. To identify θ_s^* , we evaluate the hypothetical maximum joint probability in Eq. (4), as $f_{\min}(c(\theta_s^*))$ always returns one by definition as in Eq. (2), and it does not provide the confidence that θ_s^* found so far is indeed the true optimum. However, when multiplied with $f(\theta_s)$, $f_{\min}(c(\theta_s) \mid \theta_s) f(\theta_s)$ can quantify the confidence that the found θ_s^* is the true optimal parameters, i.e., the lower the value, the more likely that θ_s^* is the true optimum.

Code (Parameter) Diffusion. After the *prior* propagation, the search space for $\mathcal{G}_s$ is defined as $\hat{\Theta}' = \{\theta_p^*\}$, where θ_p^* is considered the initial candidate for the optimal parameter for $\mathcal{G}_s$, i.e., $\theta_s^{(0)} = \theta_p^*$, and Eq. (4) is evaluated with it. to find an improved θ_s^* , we apply code diffusion to θ_p^* , which hypothetically generates the next *posterior* candidate $\theta_s^{(1)}$ from θ_p^* , and subsequently the program code $c(\theta_s^{(1)})$. At the t th optimization iteration, the *posterior* parameter $\theta_s^{(t)}$ is obtained by hypothetically diffusing θ_p^* as follows:

$$\theta_s^{(t)} = \sqrt{1 - \sigma_{s,t}^2} \theta_p^* + \sqrt{\sigma_{s,t}^2} \varepsilon \quad (5)$$

where $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$, and $\sigma_{s,t}^2 < 1$ is the variance of the diffusion for $\theta_s^{(t)}$. The newly diffused parameter $\theta_s^{(t)}$ is then appended to the search space as $\hat{\Theta}' = \{\theta_s^{(0)}, \theta_s^{(1)}, \dots, \theta_s^{(t-1)}\} \cup \{\theta_s^{(t)}\}$ and evaluated using Eq. (4). If $\theta_s^{(t)}$ yields the maximum value for Eq. (4), it is deemed the optimal, i.e., $\theta_s^* = \theta_s^{(t)}$. By repeating code diffusion, $\theta_s^{(t)}$ gradually diffuses from θ_p^* , expanding the search space from the *prior* distribution $f(\theta_p)$, in contrast to many existing approaches [3, 14] that operate within a fixed search space. By searching for θ_s^* starting from the mode of the *prior* distribution θ_p^* and iteratively diffusing θ_s around it, θ_s^* can be efficiently found within highly likely search spaces.

3.2 Empirical Support

The concept Bayesian code diffusion is based on the following empirical findings, which form the basis of its core principles.

Commonalities in Incomplete Programs (Sketches). Table 2 presents a list of subgraphs of BERT [11], indicating that five out of eight subgraphs share the same incomplete programs, i.e., the sets of optimization rules (sketches) S_0 based

on the similarity of their operations when utilizing Ansor [47]. Nevertheless, even though some subgraphs exhibit identical forms of sketches, independent program optimization tasks and distinct search spaces are assigned for each subgraph. This arises from potential differences in their details, e.g., tensor dimensions, resulting in a redundant and inefficient search process. Given that many deep learning models exhibit repetitive operations in their network architectures, there exists a potential for the proposed *prior* propagation, contingent upon the subgraphs sharing the same sketches from which program code parameters can be effectively diffused.

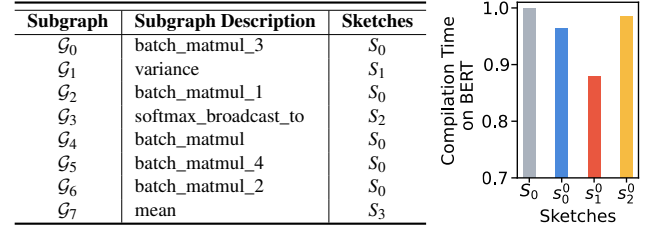


Table 2: (Left) The subgraphs in BERT [11] and their sketches obtained by applying Ansor [47], where subgraphs $\mathcal{G}_{0,2,4,5,6}$ produce the same sketches $S_0 = \{s_0^0, s_1^0, s_2^0\}$.

Figure 3: (Right) The compilation time of the subgraphs $\mathcal{G}_{0,2,4,5,6}$ taken by Ansor [47] on the operation `batch_matmul` in BERT [11] using the sketches S_0 and each sketch $s_i^0 \in S_0$ on a GPU. Here, '1.0 (gray)' denotes the time required when using all sketches $S_0 = \{s_0^0, s_1^0, s_2^0\}$.

Sketch-based Optimization. Figure 3 illustrates an example in which a set of subgraphs deriving the same sketches $S_0 = \{s_0^0, s_1^0, s_2^0\}$, i.e., $\{\mathcal{G}_0, \mathcal{G}_2, \mathcal{G}_4, \mathcal{G}_5, \mathcal{G}_6\}$, can be effectively optimized from the single sketch, s_1^0 . The bars in the figure present the optimization (compilation) time required to achieve the same program latency when using only a single sketch ($s_i^0 \in S_0$), compared to using all sketches (S_0). It is observable that rather than exhaustively exploring the parameter search space of all sketches, it is more efficient to focus on a single sketch. Thus, if one of the subgraphs sharing the same sketches identifies the optimal parameters for one of them, the others can also derive optimized parameters from the same sketch with reduced search time. This indicates that the parameter search space can be effectively reduced by optimizing a common single sketch among similar subgraphs.

Parameter Distances. Fig. 4 (left) shows the distances between the optimized configuration, i.e., sketches and their parameters, for the subgraphs that have the same incomplete programs (sketches). It is compared with the distances between the configuration for subgraphs that have different sketches (right). The figure shows that the distances within the identical sketches cluster are significantly closer compared to those across the sketches cluster. This suggests that it is more likely to identify the optimal configuration by starting from one of the optimal parameters for the same sketch and taking it as a *prior* to explore in its vicinity through code diffusion.

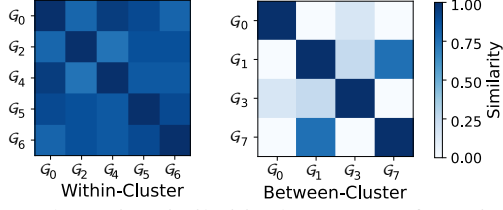


Figure 4: The cosine similarities between configurations optimized by Ansor [47]. The subgraphs $\{G_0, G_2, G_4, G_5, G_6\}$ are optimized from the same sketch, while $\{G_0, G_1, G_3, G_7\}$ have different sketches. The darker, the higher similarity. The details of distance measurement is in Fig. 10.

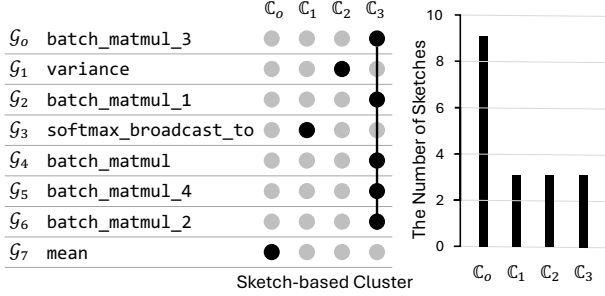


Figure 5: An example of sketch clustering applied to BERT [11], comprising four clusters: C_0 , C_1 , C_2 , and C_3 . Subgraphs within the same cluster are linked by vertical lines. As shown in Fig. 3, subgraphs with identical sketches are grouped into the same cluster. The figure on the right shows the number of sketches for each cluster. For example, C_3 consists of subgraphs that have sketches $S_0 = \{s_0^0, s_0^1, s_0^2\}$.

4 Implementation

We describe how Bayesian code diffusion (Sec. 3.1) is implemented within deep learning compilers, based on Ansor [47]. The overall procedure of Bayesian code diffusion is summarized in Alg. 1, and an example is illustrated in Fig. 6.

4.1 Sketch-based Subgraph Clustering

The first step of Bayesian code diffusion is to cluster a set of subgraphs into distinct groups. Existing approaches, such as Nimble [21], SelectiveTuning [3], and FamilySeer [26], group subgraphs based on their operations, which do not necessarily produce identical sketches due to variations such as tensor dimensions. Since different sketches correspond to distinct optimization spaces and configurations, the optimal program code may vary accordingly, thus increasing the search space to explore. For this reason, we cluster subgraphs based on their sketches to enhance both the likelihood of discovering effective prior program parameter θ_p^* and the efficiency of code diffusion, motivated by the observations provided in Sec. 3.2. The search space of *posterior* subgraphs is initialized to the parameter search space for a sketch where θ_p^* is observed, thereby reducing the overall search space efficiently. As shown in Fig. 3, sketch-based optimization not only simplifies the optimization scopes and processes but also reduces the search space, as the proposed code diffusion occurs solely

Algorithm 1 Bayesian code diffusion. $\theta^{*,i}$ denotes the optimal parameter for subgraph G^i , and $\mathcal{B}$ indicates the time budget. Hyper-parameters ϵ_e and ϵ_m define the number of iterations for fine-tuning and hardware measurement, respectively.

Input: Subgraphs $G = [G^0, \dots, G^n]$

Output: Optimal parameters θ^* for G

BayesianCodeDiffusion begin

$[G^1, \dots, G^m] \leftarrow \text{SketchBasedClustering}(G)$

for each cluster $G^i \in [G^1, \dots, G^m]$ **do**

$G_{\text{prior}} \leftarrow G_{\text{prior}} \cup \{G_{\text{prior}}^i \mid G_{\text{prior}}^i \in G^i\}$ [Eq. 6, 7]

$G_{\text{posterior}} \leftarrow G_{\text{posterior}} \cup \{G_{\text{posterior}}^{i,j} \in G^i \mid G_{\text{posterior}}^{i,j} \neq G_{\text{prior}}^i\}$

for $G_{\text{prior}}^i \in G_{\text{prior}}$ **do**

$\text{PriorOptimization}(G_{\text{prior}}^i, \epsilon_e, \epsilon_m)$

end

for $G_{\text{posterior}}^{i,j} \in G_{\text{posterior}}$ **do**

$\text{CodeDiffusion}(G_{\text{posterior}}^{i,j}, \theta^{*,i}, \frac{\epsilon_e}{2}, \frac{\epsilon_m}{2})$

end

end

while $\mathcal{B} > 0$ **do**

$G^{i,j} \leftarrow \text{GradientBasedSelection}(G)$

$\text{CodeDiffusion}(G^{i,j}, \theta^{*,i}, \frac{\epsilon_e}{2}, \frac{\epsilon_m}{2})$

end

end

PriorOptimization($G, \epsilon_e, \epsilon_m$) **begin**

$S \leftarrow \text{SketchGeneration}(G)$

$\text{RandomInitialization}(G)$

$\text{FineTuning}(G, \epsilon_e)$

$\mathcal{B} \leftarrow \mathcal{B} - \epsilon_m$

end

CodeDiffusion($G, \theta^*, \epsilon_e, \epsilon_m$) **begin**

$S \leftarrow \text{SketchGeneration}(G)$

$S \leftarrow \{s \in S \mid s \text{ is sketch for } \theta^*\}$

$\text{PriorPropagation}(G)$

$\text{FineTuning}(G, \epsilon_e)$

$\mathcal{B} \leftarrow \mathcal{B} - \epsilon_m$

end

[Fig. 8, Eq. 8, 9]

within the parameters, without involving optimization rules. This approach enables parameters to evolve efficiently during code diffusion, facilitating the automatic discovery of optimal parameters. Fig. 5 shows an example of sketch-based subgraph clustering. Nonetheless, Bayesian code diffusion can also be applied to operation-based clustering [3].

4.2 Prior Selection and Optimization

The second step of Bayesian code diffusion is to select a *prior* subgraph G_p in each cluster and optimize it. Since subgraphs in the same cluster share structurally identical operations but differ in their tensor dimensions, we evaluate their similarity based on these tensor dimensions among them. For example, the subgraphs $G_{0,2,4,5,6}$ in Fig. 3 all perform the same operation, batch_matmul, yet exhibit variations in their tensor dimensions. Then, the *prior* subgraph G_p is selected such that its tensor dimensions are closest to those of all the other

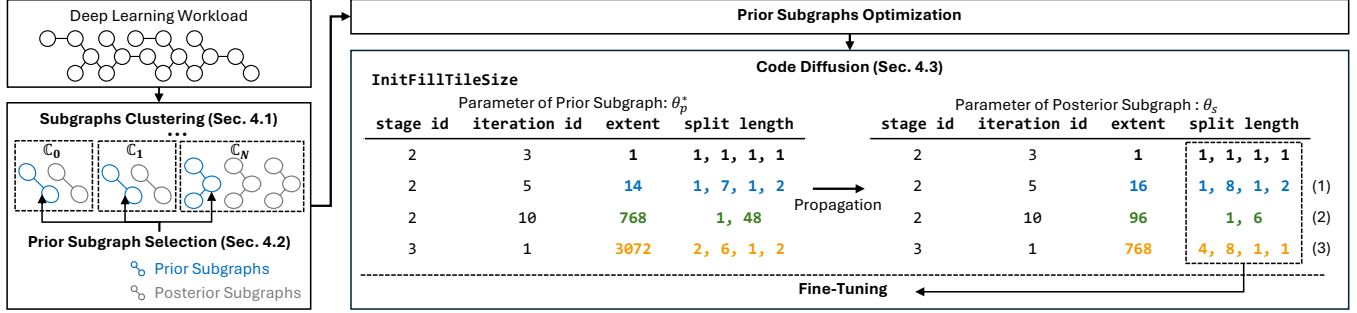


Figure 6: Given a set of subgraphs from a deep learning model, they are first clustered according to their sketches derived by Ansor [47]. Within each subgraph cluster, a prior subgraph is selected and optimized. Subsequently, the remaining subgraphs are optimized through prior parameter propagation, followed by iterative code diffusion using the Bayesian implementation.

Table 3: The examples of parameter initialization rules for program sampling used in Ansor [47] for both CPU and GPU.

CPU	GPU
InitFillTileSize	InitFillTileSize
InitUnroll	InitUnroll
InitChangeComputeLocation	
InitVectorization	
InitParallel	

subgraphs in the cluster while maintaining distinction from the subgraphs in different subgraph clusters. It is based on the assumption that the subgraph with tensor dimensions most similar to those of all other subgraphs is expected to enhance the effectiveness of code diffusion for the *posterior* subgraphs (Sec. 3.2), which also improves the code compatibility.

Given a set of N subgraphs $G = \{\mathcal{G}^0, \mathcal{G}^1, \dots, \mathcal{G}^N\}$ in a subgraph cluster, each subgraph $\mathcal{G}^n$ consists of an operator sequence $o^n = [o_0^n, o_1^n, \dots, o_K^n]$ with their tensor dimensions $[\mathbf{i}_0^n, \mathbf{i}_1^n, \dots, \mathbf{i}_j^n]$. To compose the tensor dimension vector $\mathbf{i}^n$ for o^n , $[\mathbf{i}_0^n, \mathbf{i}_1^n, \dots, \mathbf{i}_j^n]$ is concatenated as $\mathbf{i}^n = \text{concat}(\mathbf{i}_0^n, \mathbf{i}_1^n, \dots, \mathbf{i}_j^n)$. Then, cosine similarities between all pairs of $\mathbf{i}^p$ and $\mathbf{i}^q$ are computed to construct the similarity matrix $\mathbf{S} \in \mathbb{R}^{N \times N}$, where the (p, q) th element $s_{p,q}$ is defined as:

$$s_{p,q} = [\mathbf{i}^p \cdot \mathbf{i}^q / \|\mathbf{i}^p\| \cdot \|\mathbf{i}^q\|] \quad \forall 1 \leq p, q \leq N. \quad (6)$$

Given the similarity matrix $\mathbf{S}$, the *prior* subgraph $\mathcal{G}_p$ is chosen to maximize the average similarity of tensor dimensions across the elements of each row of $\mathbf{S}$, which is given by:

$$\underset{1 \leq p \leq N}{\operatorname{argmax}} \frac{1}{N} \sum_{q=1}^N s_{p,q} \quad \forall s_{p,q} \in \mathbf{S} \quad (7)$$

For example, consider three simple subgraphs, $\{\mathcal{G}_0, \mathcal{G}_1, \mathcal{G}_2\}$. The operation in $\mathcal{G}_0$ is $[[1, 1]] + [[2, 2]]$, whereas in $\mathcal{G}_1$, it is $[[1, 1], [2, 2]] + [[3, 3], [4, 4]]$. Additionally, the operation in $\mathcal{G}_2$ is defined as $[[1, 1, 1], [2, 2, 2]] + [[3, 3, 3], [4, 4, 4]]$. Although these subgraphs are structurally identical, i.e., execute same operation, add, they differ in their tensor dimensions, i.e., the tensor in $\mathcal{G}_0$ has dimension (1,2,1,2), in $\mathcal{G}_1$ dimension (2,2,2,2), and in $\mathcal{G}_2$ dimension (2,3,2,3).

Given tensor dimension similarities defined as $\text{Sim}(\mathcal{G}_0, \mathcal{G}_1) = 0.95$, $\text{Sim}(\mathcal{G}_1, \mathcal{G}_2) = 0.98$, $\text{Sim}(\mathcal{G}_2, \mathcal{G}_3) = 0.99$. We select $\mathcal{G}_2$ as the *prior* subgraph due to its higher overall similarity to other subgraphs within the cluster.

In this way, $\mathcal{G}_p$ becomes the subgraph exhibiting the highest similarity with the other subgraphs in terms of tensor dimensions. Once $\mathcal{G}_p$ is chosen, the optimal parameter θ_p^* is explored within the search space corresponding to $\mathcal{G}_p$ by evaluating Eq. (1) through random search, with θ_p sampled from a uniform distribution. This exploration is repeated in proportion to the size of the cluster, i.e., the number of subgraphs $|\mathcal{G}|$. During this process, the cost model is trained using diverse data samples generated by various *prior* subgraphs with differing configurations within each cluster, improving its potential generalization ability across clusters. We refer to this training phase as pre-training.

4.3 Code Diffusion for Posterior Optimization

The final step of Bayesian code diffusion is to propagate the *prior* parameters to the other subgraphs in the same cluster and apply code diffusion by iteratively searching for the optimal *posterior* parameters for each subgraph alongside the *prior* parameter distribution. In auto-tuning frameworks like Ansor [47], which comprises random parameter initialization and fine-tuning, code diffusion is applied to random parameter initialization, subsequently facilitating the efficient fine-tuning of sampled programs. Given a sketch, the parameter initialization rules, shown in Tab. 3, are applied for program sampling, performing optimization tasks such as tiling, parallelization, and vectorization. In existing methods [47], the parameters governing these rules are typically searched randomly, causing inefficiency in optimization, which can be accelerated by diffusing the *prior* parameter. Here, we describe how code diffusion is applied to `InitFillTileSize` in Ansor [47], which works on both CPU and GPU. The implementations for other rules are similar and thus omitted. The role of `InitFillTileSize` is to initialize the parameters of the split-step optimization rule (SP), which divides a loop (iterator) into two or more nested loops. It assigns tile sizes

Original Code	Transformed Code via SP	extent: 1024, length: 16, 64
for i in 0..1024: ...	for i_outer in 0..16: for i_inner in 0..64: ...	

Figure 7: An example code transformation using the split-step (SP) with $\text{extent}=1024$ and $\text{length}=[16, 64]$.

based on two parameters, i.e., extent and length , to fit the extent of each loop axis, as illustrated in Fig. 7. The extent refers to the length of the axis to split and length denotes the multiple split factors. Given extent , code diffusion searches for the optimal parameters for length by diffusing the *prior* parameter θ_p^* to the *posterior* parameter θ_s , as shown in Fig. 6.

Although the optimization task for each subgraph executes the split-step at the same stage and iteration, the length of the loops to be split differ among subgraphs, which complicates the application of the *prior* parameters. To enable code diffusion across loops of varying length , we devise three mechanisms of code diffusion, which implement Eq. (5) in Sec. 3.1 without knowing the exact true probability distribution $f(\theta_s) = \mathcal{N}(\theta_p^*, \sigma_s^2)$.

First, if the extent of the split-step of the *prior* subgraph $\mathcal{G}_p$ matches that of the *posterior* subgraph $\mathcal{G}_s$, the same length of θ_p^* is applied to θ_s . Conversely, if the extent differ, the length in the *prior* parameter θ_p^* is diffused to θ_s such that θ_s is closest to the length of θ_p^* among all length derivable from the extent of θ_s . In particular, by using the rule that length consists of a set of divisors of the extent , we choose the divisor of the extent of θ_s closest to length of θ_p^* . Let e_p and e_s be the extent of $\mathcal{G}_p$ and $\mathcal{G}_s$, respectively, and the length of e_p is given as $\mathbf{l}_p = \{l_p^0, \dots, l_p^n\}$. Then, the length $\bar{\mathbf{l}}_s$ of the *posterior* parameter θ_s is diffused as:

$$\bar{\mathbf{l}}_s = \{\bar{l}_s^i \in \mathbf{l}_s \mid \forall l_p^j \in \mathbf{l}_p, \bar{l}_s^i = \min |l_p^j - l_s^i|\} \quad (8)$$

where $\mathbf{l}_s = \{l_s^i \in \mathbb{Z}^+ \mid l_s^i \text{ divides } e_s\}$.

For example, the length $\mathbf{l}_p$ of $\theta_p^*: [1, 7, 1, 2]$ is diffused to the $\bar{\mathbf{l}}_s$ of $\theta_s: [1, 8, 1, 2]$, as shown (1) in Fig. 6. The blue box in Fig. 8 shows the code implementation for this case.

Second, the length $\bar{\mathbf{l}}_s$ of θ_s is diffused based on the ratio of the extent and length of θ_p^* , as follows:

$$\bar{\mathbf{l}}_s = \{\bar{l}_s^i \in \mathbf{l}_s \mid \bar{l}_s^i = \min |l_s^i - \hat{l}_s^i|, \hat{l}_s^i = \left\lceil l_p^i \cdot \frac{e_s}{e_p} \right\rceil \mid \forall l_p^i \in \mathbf{l}_p\} \quad (9)$$

which corresponds to the green box in Fig. 8. For instance, $\mathbf{l}_p$ of $\theta_p^*: [1, 48]$ is diffused to $\theta_s: [1, 6]$, as shown (2) in Fig. 6. This ensures that the length of θ_s is proportionally aligned with that of θ_p^* , thus diffusing them in close proximity to the hypothetical *prior* parameter distribution $f(\theta_p)$.

Lastly, to ensure diversity in search spaces, the length of θ_s is generated randomly, without code diffusion, as shown in the yellow box in Fig. 8 with an example given (3) in Fig. 6. Given that θ_s^* can be possibly far from θ_p^* , this diversification process should be considered in optimization.

```
InitFillTileSize::Apply(search_policy, state){
  _extent = prior ->extent;
  _lengths = prior ->lengths;

  if (_extent == extent){ lengths = _lengths;
  else {
    random_choice = random_device();
    if (random_choice == 0){
      if (extent % mul(_lengths) == 0){
        lengths = _lengths;
      } else {
        divisors_of_extent = calculate_divisors(_extent);
        lengths = map_lengths_to_closest_divisors(_lengths);
      }
      Reuse Prior  $\theta_p^*$  or diffuse  $\theta_p^*$  to  $\theta_s$  (closest to  $\theta_p^*$ )
    }

    else if (random_choice == 1){
      divisors_of_extent = calculate_divisors(_extent);
      lengths = map_lengths_to_closest_divisors(
        _lengths*(_extent/extent));
      Reuse Prior  $\theta_p^*$  or diffuse  $\theta_p^*$  to  $\theta_s$  (based on the extent)
    }

    else if (random_choice == 2){
      Randomly Select for Bias ***
      Random Selection for Bias
    }
  }
}
```

Figure 8: The implementation of for InitFillTileSize in Ansoir [47], showing three cases of code diffusion for posterior optimization (i.e., blue, green, and yellow boxes).

Among these three code diffusion methods, one of them is randomly chosen for the *posterior* subgraph optimization at every t -th search iteration. From this, the *posterior* parameter is diffused to $\{\bar{\mathbf{l}}_s\} \in \theta_s^{(t)}$ and then appended to the search space as $\hat{\Theta}' = \{\theta_s^{(0)}, \theta_s^{(1)}, \dots, \theta_s^{(t-1)}\} \cup \{\theta_s^{(t)}\}$. In this way, $f(\theta_s) = \mathcal{N}(\theta_p^*, \sigma_s^2)$ can be effectively implemented, allowing for identifying θ_s^* that maximizes Eq. (4) and quantifying the confidence whether θ_s^* is the true optimal parameter or not, i.e., the lower the value of $f_{\min}(p(\theta_s) \mid \theta_s)f(\theta_s)$, the higher it is likely to be true optimal. Fig. 9 shows an example of the *posterior* program $c(\theta_s^*)$ optimized from the *prior* program $c(\theta_p^*)$ through code diffusion.

<pre>Placeholder: p0, p1 parallel b.0@i.0@j.0@ (0,16) T_batch_matmul_NN.local auto-unroll: 64 for i.c.1 (0,7) for k.0 (0,768) for j.c.2 (0,3) for k.1 (0,4) for j.c.3 (0,32) T_batch_matmul_NN.local = ... for i.1 (0,7) for j.1 (0,96) T_batch_matmul_NN = ...</pre>	<pre>Placeholder: p0, p1 parallel b.0@i.0@j.0@ (0,48) T_batch_matmul_NN.local auto-unroll: 16 for i.c.1 (0,7) for k.0 (0,192) for j.c.2 (0,2) for k.1 (0,4) vectorize j.c.3 (0,16) T_batch_matmul_NN.local = ... for i.1 (0,7) vectorize j.1 (0,32) T_batch_matmul_NN = ...</pre>
$c(\theta_p^*)$	$c(\theta_s^*)$

Figure 9: An example of the posterior program code $c(\theta_s^*)$ (right) diffused from the prior program code $c(\theta_p^*)$ (left).

The true distribution $f(\theta_s)$ described in Sec. 3.1 is challenging to obtain directly within the program parameter space. However, it can be estimated by evaluating the distance between the mode (mean) of the *prior* distribution, θ_p^* , and the *prior* parameter θ_s . Fig. 10 illustrates the schedule encoding process used for distance measurement. The schedule comprises optimization rules (represented as text) and their corresponding parameters (represented as a list of numbers) [47]. Due to the combination of text and parameters existing within a dynamic range—caused by differences in the type and number of optimization rules across sketches—directly comparing

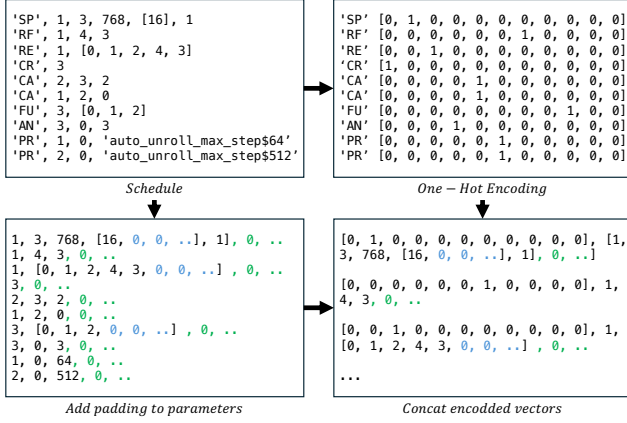


Figure 10: The process of converting a schedule [47] into a vector representation for measuring distances between schedules: the blue and green text indicate padding applied to standardize the vector’s shape.

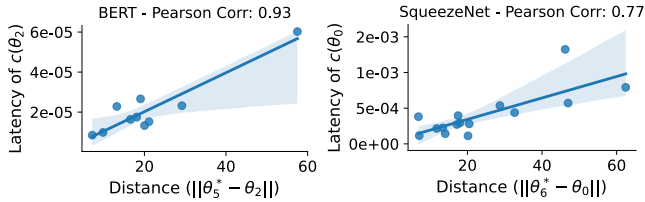


Figure 11: The latencies (ms) of the generated programs measured on CPU over the norm distance between optimal parameter θ^* and the parameter θ of another subgraph in Ansor [47] for BERT [11] (left) and SqueezeNet [18] (right).

distances or similarities between schedules becomes complex. Consequently, encoding these schedules as vectors is necessary. Here, a total of 11 optimization rules are employed, including SP (split-step), FU (fuse), among others [47]. The textual representations of these rules are converted into a one-hot vector of length 11. Subsequently, the corresponding parameters are zero-padded to transform the varying lists of numbers into a static length for uniformity. In Fig. 10, blue text denotes zero padding applied to specific parameters within identical rules, while green text represents zero padding applied to parameters across different rules. Finally, the one-hot and padded vectors are concatenated to form a unified representation. Figure 11 shows the correlation between the distance between the optimal parameter θ_a^* and θ_b of the other subgraph and the latency. This figure demonstrates that the program latency $l(c(\theta_b))$ decreases as the distance between θ_b and θ_a^* diminishes, indicating that a more optimal program can be discovered when θ_b is diffused from θ_a^* .

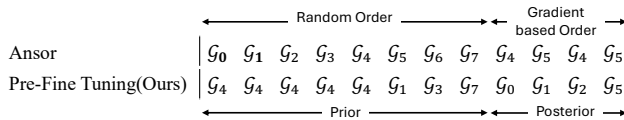


Figure 12: The optimization sequence of BERT [11] subgraphs of Ansor [47] and Bayesian code diffusion (Pre-Fine Tuning), which trains the cost model with datasets generated in different orders, with subgraphs optimized by Ansor.

5 Cost Model of Bayesian Code Diffusion

In auto-tuning, the cost model is essential for evaluating candidate programs by filtering out inefficient ones. Surviving candidates are executed on hardware, generating execution results used to refine the model’s prediction accuracy. By continuously learning from online execution results, the cost model gradually improves its predictive performance from scratch within the online learning paradigm. With an accurate cost model, program execution can be selectively guided, substantially reducing the need for exhaustive hardware measurements and accelerating the compilation time. For example, Tenset [48] has demonstrated that a cost model pre-trained on a large-scale dataset can accelerate search time.

Two studies highlight the importance of the cost model from different perspectives. When the training data for the cost model is biased by the search method, the stability of its predictive accuracy can deteriorate [28], while Family-Seer [26] deliberately biases the data by grouping subgraphs and training a dedicated cost model for each group, allowing faster convergence and program selection. They show that prediction performance and learning efficiency can be improved depending on the learning strategy and cost model.

In optimizing process of *prior* and *posterior* subgraphs, the cost model of Bayesian code diffusion leverages strengths of the two approaches. First, during the optimization of *prior* subgraphs within each cluster, the cost model is trained using the execution data from *prior* programs across diverse clusters. We refer to this process as pre-training. Then, by tuning the *posterior* subgraphs, the cost model is further refined with execution data from *posterior* programs similar to their corresponding *prior* programs. We refer to this process as fine-tuning. This learning strategy enables the cost model to converge quickly while minimizing data bias.

Importantly, we do not modify the cost model architecture or input features and use XGBoost [4] as the cost model, following Ansor [47]. Instead, we adjust the sequence in which execution (training) data is generated. Fig. 12 depicts subgraph optimization sequences of Ansor [47] and Bayesian code diffusion, generating data in different orders for cost model training. As shown in Fig. 5, for the clusters $\mathbb{C}_0, \mathbb{C}_1, \mathbb{C}_2$ and $\mathbb{C}_3$, *prior* subgraphs are selected as $\mathcal{G}_7, \mathcal{G}_3, \mathcal{G}_1$ and $\mathcal{G}_4$, respectively. Bayesian code diffusion optimizes each *prior* in proportion to the size of the cluster it belongs to, assigning greater importance to larger clusters. This contrasts with Ansor’s cost model training sequence, which begins with a round-robin over randomly ordered subgraphs and then proceeds with a gradient-based selection strategy.

6 Experiments

We implement the proposed Bayesian code diffusion on Ansor [47] (tvm@a340dbe), which is available at GitHub

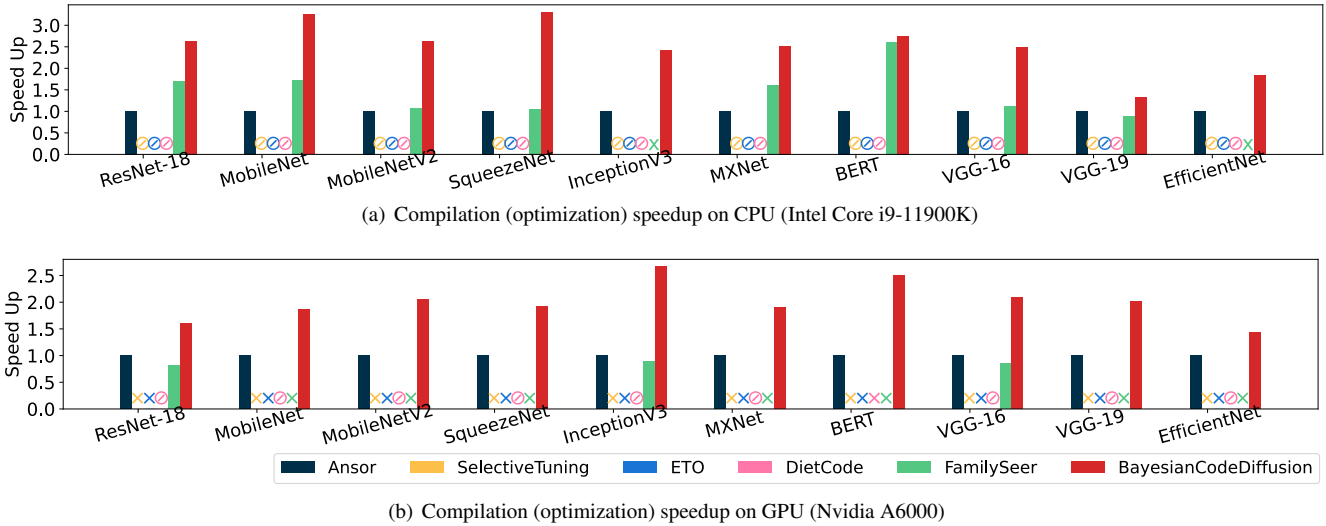


Figure 13: The compilation (optimization) time speedup in generating deep learning programs with execution latencies equivalent to the best latencies achieved by Anso [47], normalized to a baseline of 1.0 (the blue bars). The symbol ‘x’ indicates that the corresponding method fails to generate a program with latency equivalent to Anso’s best, while ‘Ø’ denotes that the corresponding method is not applicable to the target deep learning model or hardware (CPU/GPU).

repository¹, providing the detailed procedures. We evaluate it on an Intel Core i9-11900K@3.50GHz CPU and Nvidia A6000 GPU with a wide range of deep learning models, i.e., ResNet-18 [16], VGG-{16,19} [38], BERT [11], MobileNet [17], MobileNet-V2 [36], SqueezeNet-V1.1 [18], Inception-V3 [39], MXNet [5], and EfficientNet [40].

6.1 End-to-end Model Compilation

Fig. 13 presents the end-to-end optimization (compilation) speedup achieved by Bayesian code diffusion in generating deep learning programs with execution latencies equivalent to the best-latency programs produced by Anso [47] on both CPU and GPU. The results are compared with alternative approaches, i.e., Anso [47], FamilySeer [26], DietCode [46], ETO [12], and SelectiveTuning [3]. On the CPU, the proposed Bayesian code diffusion outperforms Anso [47] by an average of 2.52 $\times$ speedup (arithmetic mean) and other methods by an average of 1.95 $\times$ speedup, with a maximum speedup of up to 3.31 $\times$. Similarly, on the GPU, Bayesian code diffusion outperforms all other methods by an average of 2.00 $\times$, with a maximum speedup of up to 2.79 $\times$. The result also shows that previous methods are either (1) unable to generate programs with the equivalent execution latencies, such as ETO [12] and FamilySeer [26], which are marked as x in Fig. 13 or (2) inapplicable to specific subgraphs (operators) or hardware platforms (CPU or GPU), such as DietCode [46], ETO [12], and SelectiveTuning [3] marked as Ø. In particular, DietCode [46] is designed with GPU architectural features, such as thread blocks, and thus is limited to supporting only GPUs and a subset of operators, rather than the full range of

operators and CPUs. Notably, DietCode does not implement the support for convolutional operations, and consequently, it is only evaluated on BERT. Similarly, ETO [12] is also tailored for GPU architectures, and SelectiveTuning [3] has demonstrated effectiveness only on GPUs, with no reported evaluation conducted on CPUs. Unlike these existing methods, which lack applicability, compatibility, optimality, and hardware independence, the proposed Bayesian code diffusion consistently generates programs with the minimum optimization times across various models and hardware (both CPU and GPU), enabling reliable, versatile, and efficient deep learning program optimization.

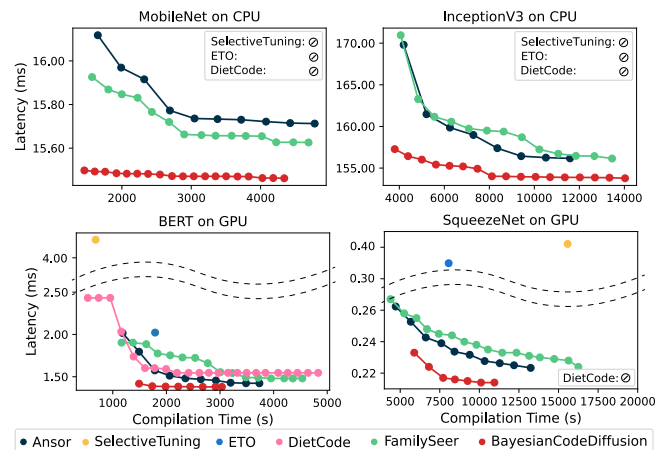


Figure 14: The execution latencies (ms) of the generated programs over compilation (optimization) time (s). The methods that are not applicable either to the target deep learning models or hardware (CPU/GPU) are indicated by the symbol ‘Ø’.

Fig. 14 presents program execution latencies over compila-

¹<https://github.com/eai-lab/BayesianCodeDiffusion>

Table 4: The execution latency speedups of the programs generated by Bayesian code diffusion: the first diffused latency (left of the slash), measured at the first iteration, and the last diffused latency (right of the slash), measured after all iterations are completed. These are compared against alternative approaches, with Ansor’s optimal latency normalized to 1.0. The symbol ‘ $\emptyset$ ’ denotes that the method is not applicable to the target deep learning model or hardware (CPU/GPU).

Hardware	Method	Model									
		ResNet-18	MobileNet	MobileNetV2	SqueezeNet	InceptionV3	MXNet	BERT	VGG-16	VGG-19	EfficientNet
CPU	SelectiveTuning	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
	ETO	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
	DietCode	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
	FamilySeer	1.02 / 1.01	1.01 / 1.01	1.02 / 1.01	1.07 / 1.01	1.00 / 1.01	1.02 / 1.00	1.02 / 1.01	0.99 / 1.00	1.00 / 1.01	1.00 / 1.00
	Bayesian code diffusion	1.04 / 1.01	1.04 / 1.02	1.08 / 1.02	1.21 / 1.03	1.16 / 1.01	1.05 / 1.01	1.02 / 1.01	1.04 / 1.01	1.00 / 1.00	1.03 / 1.01
GPU	SelectiveTuning	0.38 / 0.28	0.60 / 0.44	0.58 / 0.40	0.68 / 0.56	0.83 / 0.56	1.03 / 0.58	0.47 / 0.32	0.71 / 0.52	0.42 / 0.28	0.94 / 0.72
	ETO	0.94 / 0.68	0.55 / 0.41	0.63 / 0.43	0.80 / 0.65	1.18 / 0.80	0.78 / 0.44	1.04 / 0.70	0.87 / 0.63	0.69 / 0.47	1.04 / 0.80
	DietCode	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	0.86	$\emptyset$	$\emptyset$	$\emptyset$
	FamilySeer	1.00 / 1.02	0.99 / 0.98	1.03 / 0.98	1.00 / 1.00	1.00 / 1.01	1.02 / 0.98	1.10 / 0.97	1.00 / 1.01	1.00 / 0.93	1.02 / 0.99
	Bayesian code diffusion	1.26 / 1.06	1.29 / 1.07	1.36 / 1.05	1.15 / 1.05	1.39 / 1.08	1.65 / 1.07	1.47 / 1.03	1.16 / 1.07	1.37 / 1.13	1.20 / 1.01

tion (optimization) time on both CPU and GPU. It shows that Bayesian code diffusion provides enhanced performance in both program latency and compilation time when compared to alternative methods. For example, it provides optimization (compilation) speedups of $3.25\times$ for MobileNet on CPU, $2.28\times$ for InceptionV3 on CPU, $2.50\times$ for BERT on GPU, and $1.91\times$ for SqueezeNet on GPU, while generating programs whose execution latencies are consistently lower than those of other methods throughout the entire optimization. This is because Bayesian code diffusion and existing approaches, such as Ansor [47], explore the search space with distinct strategies. Notably, Bayesian code diffusion explicitly targets narrowed and more probable subsets of the search space. Although other approaches could find programs with comparable execution latencies to Bayesian code diffusion by continuously exploring the large entire search space, they would come at the cost of significantly increased time. As a result, Bayesian code diffusion can efficiently constrain and explore the search space, and identifies lower-latency programs more effectively compared to alternative methods, under the same time budget.

Tab. 4 summarizes the program latency speedup achieved by Bayesian code diffusion at the first iteration (i.e., first-diffused latency) and the last iteration of code diffusion (i.e., last diffused latency), respectively, in comparison with alternative methods. Following Ansor [47], Bayesian code diffusion allocates the total search time budget by dividing it among the subgraphs. After each subgraph is optimized through the pre-fine tuning process described in Sec. 5, the remaining time budget is used to further optimize the subgraphs in a gradient-based order [47]. We define the first-diffused latency as the program latency measured when all subgraphs have been tuned at least once. The last-diffused latency refers to the program latency obtained after fully utilizing the entire time budget to optimize all subgraphs. The first-diffused latency is compared against the program latency obtained when all subgraphs are tuned by each method. In contrast, the last-diffused latency is compared against the program latency achieved af-

ter each method has exhausted its entire search time budget. At the outset (Tab. 4, left of the slash), it begins from first diffused programs exhibiting the lowest latency across all cases, for instance $1.65\times$ latency speedup for MXNet on GPU, compared to other approaches that either fail to generate programs with equivalent latencies or require significantly more time to achieve them. After full iterations of Bayesian code diffusion (Tab. 4, right of the slash), Bayesian code diffusion produces last diffused programs with the lowest program latency among all methods, e.g., $1.13\times$ speedup for VGG-19 on GPU, while significantly reducing optimization time at the same time, as presented in Fig. 13. These results suggest that a series of mechanisms of Bayesian code diffusion, i.e., *prior* optimization, and code diffusion for *posterior* program generation, brings consistent improvements in both program compilation (optimization) time and the execution latency of the generated program.

6.2 Subgraph Cluster Optimization

Tab. 5 lists a set of operator sequences of several subgraph clusters in some deep learning models, and Fig. 15 presents their optimization speedup using Bayesian code diffusion, compared with Ansor [47]. As illustrated in the figure, the proposed Bayesian code diffusion optimizes deep learning operators to achieve latencies equivalent to Ansor’s best results, with an average optimization speedup of $2.11\times$. Fig. 16 shows the optimization efficiency of Bayesian code diffusion on several subgraph clusters (i.e., clusters A, B, C, and D) over the optimization time, indicating that it generates programs with lower execution latency using reduced subgraph compilation (optimization) time compared to that of Ansor.

Table 5: A set of operation sequences in subgraphs.

Idx	Operator Sequence	Model	HW
A	conv2d_add_nn_relu	MobileNet	CPU
B	conv2d_NCHWc_add_add_nn_relu	MXNet	CPU
C	conv2d_NCHWc_add_nn_relu	InceptionV3	CPU
D	conv2d_add_nn_relu	SqueezeNet	GPU

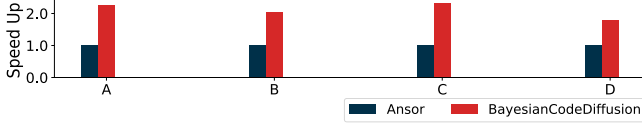


Figure 15: The optimization speedups on subgraph clusters in Tab. 5, compared against Ansor [47].

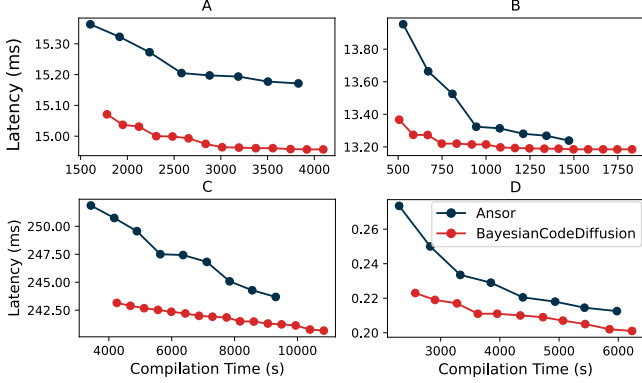


Figure 16: The latencies of the generated programs (ms) over the subgraph cluster compilation (optimization) time (s).

6.3 Cost Model

Fig. 17 plots the program execution latency over the compilation (optimization) time, comparing the proposed pre-training strategy and fine-tuning with Ansor [47] that does not apply it. The sequences of subgraph optimization for both cases are illustrated in Fig. 12, which generates training data samples in different orders for training the cost model. To evaluate the cost model as an independent variable, we optimize the programs using Ansor without using code diffusion in both experiments. The cost model is then trained on data samples produced in different orders. As shown in the figure, the proposed pre-training and fine-tuning method reduces compilation time by producing programs with lower latencies, without requiring any modifications to the cost model. This result suggests that the effectiveness of the cost model can be enhanced by first improving its generalization ability with various datasets and subsequently learning and making predictions on specific subgraph clusters. We expect further improvements could be achievable by incorporating advancements in enhancing the learning capacity and prediction accuracy of the cost model [2, 35, 44] into the proposed Bayesian code diffusion. We leave this exploration for future work.

6.4 Subgraph Sparsity

The performance of Bayesian code diffusion can be influenced by the degree of similarity among subgraphs of models.

In particular, if each subgraph generates a distinct sketch, it leads to an excessively large number of small clusters. As a result, the selected *prior* subgraph receives fewer tuning opportunities, and fewer *posterior* subgraphs can benefit from *prior* propagation, possibly reducing its effectiveness. Additionally, high operator diversity can negatively affect the cost

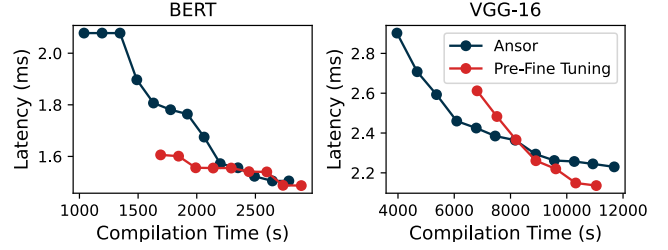


Figure 17: The latency (ms) of generated programs for BERT [11] and VGG-16 [38] over the compilation (optimization) time (s), under different training orders of the cost model as illustrated in Fig. 12.

Table 6: A comparison between sketch sparsity ($\mathbb{S}_S$), operator sparsity ($\mathbb{S}_O$), and program optimization speedup.

Network	$\mathbb{S}_O$	CPU		GPU	
		$\mathbb{S}_S$	SpeedUp	$\mathbb{S}_S$	SpeedUp
ResNet-18	0.83	0.83	2.63	0.83	1.60
MobileNet	0.14	0.57	3.25	0.14	1.85
MobileNetV2	0.44	0.76	2.62	0.71	2.05
SqueezeNet	0.22	0.43	3.30	0.22	1.92
InceptionV3	0.22	0.53	2.42	0.22	2.67
MXNet	0.62	0.86	2.51	0.62	1.91
BERT	0.38	0.50	2.75	0.50	2.50
VGG-16	0.40	0.93	2.48	0.87	2.08
VGG-19	0.40	0.93	1.33	0.87	2.01
EfficientNet	0.66	0.88	1.85	0.88	1.43

model’s pre-training and fine-tuning strategies. Even when sketches differ, subgraphs that share the same operator tend to produce similar program data. Consequently, fine-tuning results from one cluster can positively influence others that, despite having different sketches, share the same operator. However, when operator diversity is high, such cross-cluster generalization becomes limited, reducing the effectiveness of the cost model’s learning strategy.

To quantify the diversity of sketch generation and operator across subgraphs, we define sketch sparsity and operator sparsity as shown in Eq. (10), where G_{all} is the total number of subgraphs, $\max(|G_S^i|)$ is the number of subgraphs that belong to the largest cluster obtained via sketch-based clustering, and $\max(|G_O^i|)$ is the number of subgraphs containing the most frequent operator in the model. A higher sparsity value indicates lower subgraph similarity. Tab. 6 compares sketch sparsity, operator sparsity, and optimization speedup of Bayesian code diffusion in Fig. 13 across different models.

$$\mathbb{S}_S = 1 - \max(|G_S^i|)/G_{\text{all}}, \quad \mathbb{S}_O = 1 - \max(|G_O^i|)/G_{\text{all}} \quad (10)$$

Fig. 18 presents the Pearson correlation [13] between subgraph sparsity (i.e., sketch sparsity and operator sparsity) and average program optimization speedup on various models on CPU and GPU. Overall, the sketch sparsity shows a stronger correlation with optimization speedup on CPU, while operator sparsity is more strongly correlated on GPU. This indicates that *prior* propagation has a large impact on compilation time on CPU, whereas pre-training and fine-tuning strategy in cost model are more impactful on GPU.

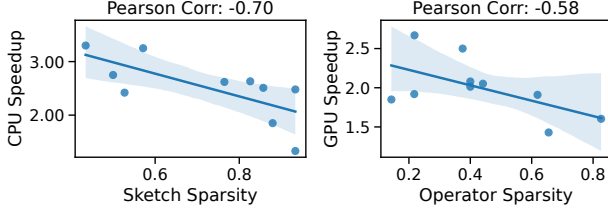


Figure 18: The Pearson correlation [13] between subgraph sparsity and speedup on CPU and GPU. On CPUs, speedup is correlated more strongly with sketch sparsity, whereas on GPUs, it is more closely associated with operator sparsity.

6.5 Prior Selection

We evaluate the *prior* selection described in Sec. 4.2. Specifically, we compare the program execution latency of the first diffused code in Bayesian code diffusion; the *prior* subgraph is selected based on either high or low similarity to other subgraphs in the same cluster. Fig. 19 presents the mean latency of l_h and l_l across various models in Fig. 13, where l_h and l_l represent the program execution latency of the first diffused code obtained by selecting *prior* subgraphs with high and low similarity, respectively. In general, selecting a *prior* subgraph with high similarity enables Bayesian code diffusion to find lower-latency programs across both CPU and GPU. On average, the normalized latency l_h/l_l is 0.99 on CPU and 0.91 on GPU, indicating consistent performance improvement of high-similarity *prior* subgraph over low-similarity *prior* subgraph. Therefore, Bayesian code diffusion tends to be more effective when selecting the subgraph with the highest similarity to others as the *prior* subgraph within the cluster.

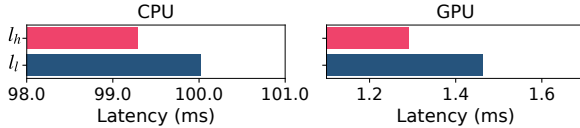


Figure 19: A comparison of l_h and l_l on CPU and GPU, where l_h and l_l denote the program execution latency of the first diffused code optimized using high- and low-similarity *prior* subgraphs, respectively. The bar lengths indicate the mean value of execution latencies of various deep learning models.

7 Discussions and Limitations

From Hypothetical Concept to Implementation. Applying the proposed concept of Bayesian code diffusion to the deep learning program optimization problem requires several considerations and presents some complexities in implementation. For instance, the true prior distribution $f(\theta_p) = \mathcal{N}(\theta_p^*, \sigma_p^2)$ is unknown and must be effectively approximated to conduct precise code diffusion for the *posterior* optimization. Although the optimal *prior* parameter θ_p^* is empirically estimated in our implementation by allocating a large search time budget proportional to the cluster size for sufficient search, there is no guarantee that the identified θ_p^* is truly optimal.

While Eq. (4) in Sec. 3.1 provides a confidence measurement on the *posterior* optimal parameter θ_s^* , a similar quantity needs to be devised for the optimal *prior* parameter θ_p^* to improve the overall performance of subsequent code diffusion. Furthermore, implementing code diffusion for the *posterior* program optimization may not be straightforward in practical implementations. In this work, we demonstrate a concrete implementation of code diffusion for Ansor [47]; however, different deep learning compilers or optimization mechanisms may require modifications of the code diffusion approach, distinct from the approach presented here. We will investigate to bridge the gap between the hypothetical concept of Bayesian code diffusion and its practical implementations, aiming to develop a generalized solution that can address or mitigate these implementation challenges.

Prior Selection. Selecting the optimal *prior* subgraph within a subgraph cluster during online program optimization is a challenging task. That is because the *prior* subgraph capable of generating the optimal programs can only be identified after the full compilation (optimization) of the target deep learning model. In this work, we estimate the most suitable *prior* during the optimization process by measuring the tensor shape distances between subgraphs in a cluster in an online manner. Although this approach results in substantial improvements in both optimization time and program execution latency, we have observed that superior *prior* subgraphs exist in certain cases, which could further enhance program optimization performance. We will study alternative approaches that can more accurately select the *prior* subgraph to facilitate enhanced code diffusion for the *posterior* parameter optimization.

Cost Model. While Bayesian code diffusion proposes and employs pre-training and fine-tuning of the cost model, which enhances the program optimization performance, additional improvements could be achieved by enhancing the cost model itself. For instance, in this work, the cost model or learning algorithm is utilized as presented in Ansor [47] without modification. However, more advanced cost models, such as neural network-based approaches [44], could be employed to enhance the model prediction accuracy. Regarding learning strategies, active learning [2], which selects and focuses on learning from data examples expected to contribute to model performance, can be used to enhance both prediction accuracy and training efficiency. Additionally, we can leverage a large pre-collected dataset as pre-training data in an offline manner [35], followed by fine-tuning the cost model in an online context. Therefore, it is worth investigating how improvements to the cost model, including modeling methods and learning strategies, affect the performance of the proposed Bayesian code diffusion in a synergistic manner. However, we generally conjecture that improvements to the cost model could help further enhance program optimization, as these mechanisms are orthogonal to one another.

Applicability to Other Deep Learning Compilers. The core idea behind Bayesian code diffusion is that structurally

similar subgraphs can share optimization patterns, enabling more efficient program tuning. This insight can extend beyond Ansor [47], to other deep learning program optimization frameworks such as MetaSchedule [37] and TASO [20], expanding its applicability. MetaSchedule [37], a stochastic search framework, probabilistically samples transformation rule parameters. Bayesian code diffusion can be integrated into MetaSchedule, where *posterior* program distributions follow *prior* distributions, or their parameters are diffused from *prior* programs. For TASO [20], a superoptimizer based on graph substitutions performs graph transformations independently by generating and comparing candidates. While TASO does not exploit subgraph similarity, its search space can be reduced by identifying and matching common subpatterns.

8 Related Work

Subgraph Similarity. Similar to Bayesian code diffusion, some existing works group subgraphs. TransferTuning [14] transfers an optimized schedule from one model to another running on CPU based on model similarity. However, it requires a pre-optimized schedule of a separate model, which differs from the proposed approach not requiring it. Instead, the *prior* is propagated in an online manner within the same model, which is applicable to both CPU and GPU. Family-Seer [26] groups subgraphs with identical operator sequences and assigns a cost model to each. However, the narrow dataset generated by a single group often leads to suboptimal program performance. The proposed approach addresses this by pre-training and fine-tuning the cost model, accelerating program optimization while ensuring optimal performance. The proposed approach can also be applied to reduce search spaces in dynamic tensor program optimizations, e.g., Nimble [21] and DietCode [46], which construct a shape-generic search space. SelectiveTuning [3] also clusters subgraphs and transfers schedules from representative subgraph to others. In contrast to this method, which provides suboptimal performance as it modifies the transferred schedule only once using pre-defined rules, the proposed approach employs iterative and adaptive code diffusion, producing optimal programs.

Efficient Search Space Exploration. Many studies try to facilitate efficient exploration of program search spaces. Being orthogonal to the proposed Bayesian code diffusion, they can be applied to further accelerate optimization in the reduced search space derived from the *prior* programs. For example, Chameleon [1] explores search spaces using reinforcement learning, and ALT [42] minimizes hardware measurements while exploring optimal compile time with active learning. Dynamic gradient descent search [24] improves the performance of the program by replacing traditional random sampling. Some works aim to address the inefficiencies of auto-schedulers. DynaTune [45] efficiently allocates time resources to optimization tasks, and AdaTune [25] reduces the hardware measurements needed to stabilize the performance

of the program. However, these approaches do not utilize similarity between subgraphs, missing opportunities for more efficient exploration and program reuse. Thus, Bayesian code diffusion can complement them. Thus, integrating Bayesian code diffusion can reduce redundant searches by reducing exploration spaces, improving exploration efficiency, reducing hardware measurements, and enhancing cost model accuracy, leading to more effective deep learning program optimization.

Cost Model. Cost models are typically trained online during compilation using data generated via hardware measurements [6, 47]. BALTO [2] reduces training data through diversity-based active learning, and TLP [44] uses a neural network and extracts features from primitives to ensure portability across various hardware and efficiently explore programs with limited data. In contrast, offline learning utilizes a pre-trained cost model using pre-collected data to reduce compile time. This includes approaches such as Tenset [48] and One-shot tuner [35] that train a neural network cost predictor offline. Although offline learning reduces compile time, it requires a substantial amount of data to train the model, which may not be available for heterogeneous hardware. This contrasts with the proposed approach for online training. In this work, we introduce the concept of online pre-training of the cost model using the *prior*s of different clusters, which is fine-tuned on cluster-specific data, one at a time. It enables the model to ensure generalizability while improving the predictions for each cluster, without modifying the cost model.

Large Language Models. Advances in large language models (LLMs) have prompted their adoption in code generation tasks [8, 9, 43]. In particular, TLM [43] proposes a tensor language model for deep learning program optimization and departs from traditional program sampling methods by reconstructing the search space through a generative compiler based on LLMs. However, this approach requires extensive computational resources due to the pre-training of huge models like GPT-2 [31] with large-scale datasets. Additionally, this offline learning paradigm struggles with heterogeneous hardware, differing fundamentally from the online learning approach adopted in Bayesian code diffusion.

9 Conclusion

We introduce Bayesian code diffusion, which enhances the auto-tuning efficiency for deep learning (tensor) programs by reformulating optimization as a distribution-based problem using Bayesian principles. By diffusing well-optimized program code as a *prior* across similar subgraphs, it enables an efficient search for optimal parameters for *posterior* programs compared to random search methods. Additionally, pre-training and fine-tuning the cost model improves prediction accuracy. Implemented on Ansor, the proposed Bayesian code diffusion accelerates compilation times and reduces the program latency for deep learning models on both CPUs and GPUs, outperforming state-of-the-art auto-tuning methods.

Acknowledgments

This work was supported by the Institute of Information & communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No.RS-2024-00508465) and Institute of Information & communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No.RS-2020-II201336, Artificial Intelligence Graduate School Program(UNIST)).

References

- [1] Byung Hoon Ahn, Prannoy Pilligundla, Amir Yazdanbakhsh, and Hadi Esmaeilzadeh. Chameleon: Adaptive code optimization for expedited deep neural network compilation. *arXiv preprint arXiv:2001.08743*, 2020.
- [2] Jun Bi, Xiaqing Li, Qi Guo, Rui Zhang, Yuanbo Wen, Xing Hu, Zidong Du, Xinkai Song, Yifan Hao, and Yunji Chen. Balto: fast tensor program optimization with diversity-based active learning. In *The Eleventh International Conference on Learning Representations*, 2022.
- [3] CH Yu. SelectiveTuning. <https://github.com/apache/tvm/issues/4188>, 2019.
- [4] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.
- [5] Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems. *arXiv preprint arXiv:1512.01274*, 2015.
- [6] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594, 2018.
- [7] Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. Learning to optimize tensor programs. *Advances in Neural Information Processing Systems*, 31, 2018.
- [8] Chris Cummins, Volker Seeker, Dejan Grubisic, Mostafa Elhoushi, Youwei Liang, Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Kim Hazelwood, Gabriel Synnaeve, et al. Large language models for compiler optimization. *arXiv preprint arXiv:2309.07062*, 2023.
- [9] Chris Cummins, Volker Seeker, Dejan Grubisic, Baptiste Roziere, Jonas Gehring, Gabriel Synnaeve, and Hugh Leather. Meta large language model compiler: Foundation models of compiler optimization. *arXiv preprint arXiv:2407.02524*, 2024.
- [10] Scott Cyphers, Arjun K Bansal, Anahita Bhiwandiwalla, Jayaram Bobba, Matthew Brookhart, Avijit Chakraborty, Will Constable, Christian Convey, Leona Cook, Omar Kanawi, et al. Intel ngraph: An intermediate representation, compiler, and executor for deep learning. *arXiv preprint arXiv:1801.08058*, 2018.
- [11] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, 2019.
- [12] Jingzhi Fang, Yanyan Shen, Yue Wang, and Lei Chen. Eto: accelerating optimization of dnn operators by high-performance tensor program reuse. *Proceedings of the VLDB Endowment*, 15(2):183–195, 2021.
- [13] David Freedman, Robert Pisani, and Roger Purves. Statistics (international student edition). *Pisani, R. Purves, 4th edn. WW Norton & Company, New York*, 2007.
- [14] Perry Gibson and José Cano. Transfer-tuning: Reusing auto-schedules for efficient tensor program code generation. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*, pages 28–39, 2022.
- [15] Google. Xla - high level optimizer. GitHub repository, 2024.
- [16] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [17] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017.
- [18] Forrest N Iandola, Song Han, Matthew W Moskewicz, Khalid Ashraf, William J Dally, and Kurt Keutzer. Squeezenet: Alexnet-level accuracy with 50x fewer parameters and <0.5 mb model size. *arXiv preprint arXiv:1602.07360*, 2016.

- [19] Intel Corporation. *Intel® Math Kernel Library (Intel® MKL)*, 2024. Version 2024.0.
- [20] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. Taso: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 47–62, 2019.
- [21] Woosuk Kwon, Gyeong-In Yu, Eunji Jeong, and Byung-Gon Chun. Nimble: Lightweight and parallel gpu task scheduling for deep learning. *Advances in Neural Information Processing Systems*, 33:8343–8354, 2020.
- [22] Chuck L Lawson, Richard J. Hanson, David R Kincaid, and Fred T. Krogh. Basic linear algebra subprograms for fortran usage. *ACM Transactions on Mathematical Software (TOMS)*, 5(3):308–323, 1979.
- [23] Chris Leary and Todd Wang. Xla: Tensorflow, compiled, 2017.
- [24] Chendi Li, Yufan Xu, Sina Mahdipour Saravani, and Ponnuswamy Sadayappan. Accelerated auto-tuning of gpu kernels for tensor computations. In *Proceedings of the 38th ACM International Conference on Supercomputing*, pages 549–561, 2024.
- [25] Menghao Li, Minjia Zhang, Chi Wang, and Mingqin Li. Adatune: Adaptive tensor program compilation made efficient. *Advances in Neural Information Processing Systems*, 33:14807–14819, 2020.
- [26] Mingzhen Li, Hailong Yang, Shanjuan Zhang, Fengwei Yu, Ruihao Gong, Yi Liu, Zhongzhi Luan, and Depei Qian. Exploiting subgraph similarities for efficient auto-tuning of tensor programs. In *Proceedings of the 52nd International Conference on Parallel Processing*, pages 786–796, 2023.
- [27] Jonas Mockus and Jonas Mockus. *The Bayesian approach to local optimization*. Springer, 1989.
- [28] Pranati Modumudi, Xiyu Zhou, and Sunghyun Park. Unveiling source of performance variance on search-based compiler optimization. In *Machine Learning for Computer Architecture and Systems 2022*.
- [29] NVIDIA Corporation. *cuBLAS: The NVIDIA CUDA Basic Linear Algebra Subroutines library*, 2024.
- [30] NVIDIA Corporation. NVIDIA cuDNN. <https://developer.nvidia.com/cudnn>, 2024. Version X.Y.Z.
- [31] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [32] Sivaramakrishnan Rajaraman, Ghada Zamzmi, Feng Yang, Zhaohui Liang, Zhiyun Xue, and Sameer Antani. Semantically redundant training data removal and deep model classification performance: A study with chest x-rays. *arXiv preprint arXiv:2309.09773*, 2023.
- [33] Jared Roesch, Steven Lyubomirsky, Marisa Kirisame, Logan Weber, Josh Pollock, Luis Vega, Ziheng Jiang, Tianqi Chen, Thierry Moreau, and Zachary Tatlock. Relay: A high-level compiler for deep learning. *arXiv preprint arXiv:1904.08368*, 2019.
- [34] Nadav Rotem, Jordan Fix, Saleem Abdulrasool, Garret Catron, Summer Deng, Roman Dzhabarov, Nick Gibson, James Hegeman, Meghan Lele, Roman Levenstein, et al. Glow: Graph lowering compiler techniques for neural networks. *arXiv preprint arXiv:1805.00907*, 2018.
- [35] Jaehun Ryu, Eunhyeok Park, and Hyojin Sung. One-shot tuner for deep learning compilers. In *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction*, pages 89–103, 2022.
- [36] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4510–4520, 2018.
- [37] Junru Shao, Xiyu Zhou, Siyuan Feng, Bohan Hou, Ruihang Lai, Hongyi Jin, Wuwei Lin, Masahiro Masuda, Cody Hao Yu, and Tianqi Chen. Tensor program optimization with probabilistic programs. *Advances in Neural Information Processing Systems*, 35:35783–35796, 2022.
- [38] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
- [39] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2818–2826, 2016.
- [40] Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*, pages 6105–6114. PMLR, 2019.
- [41] Pradnya A Vikhar. Evolutionary algorithms: A critical review and its future prospects. In *2016 International conference on global trends in signal processing, information computing and communication (ICGTSPICC)*, pages 261–265. IEEE, 2016.

- [42] Xi Zeng, Tian Zhi, Zidong Du, Qi Guo, Ninghui Sun, and Yunji Chen. Alt: Optimizing tensor compilation in deep learning compilers with active learning. In *2020 IEEE 38th International Conference on Computer Design (ICCD)*, pages 623–630, 2020.
- [43] Yi Zhai, Sijia Yang, Keyu Pan, Renwei Zhang, Shuo Liu, Chao Liu, Zichun Ye, Jianmin Ji, Jie Zhao, Yu Zhang, and Yanyong Zhang. Enabling tensor language model to assist in generating High-Performance tensor programs for deep learning. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 289–305, Santa Clara, CA, July 2024. USENIX Association.
- [44] Yi Zhai, Yu Zhang, Shuo Liu, Xiaomeng Chu, Jie Peng, Jianmin Ji, and Yanyong Zhang. Tlp: A deep learning-based cost model for tensor program tuning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 833–845, 2023.
- [45] Minjia Zhang, Menghao Li, Chi Wang, and Mingqin Li. Dynatune: Dynamic tensor program optimization in deep neural network compilation. In *International Conference on Learning Representations*, 2021.
- [46] Bojian Zheng, Ziheng Jiang, Cody Hao Yu, Haichen Shen, Joshua Fromm, Yizhi Liu, Yida Wang, Luis Ceze, Tianqi Chen, and Gennady Pekhimenko. Dietcode: Automatic optimization for dynamic tensor programs. *Proceedings of Machine Learning and Systems*, 4:848–863, 2022.
- [47] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, et al. Ansor: Generating {High-Performance} tensor programs for deep learning. In *14th USENIX symposium on operating systems design and implementation (OSDI 20)*, pages 863–879, 2020.
- [48] Lianmin Zheng, Ruochen Liu, Junru Shao, Tianqi Chen, Joseph E Gonzalez, Ion Stoica, and Ameer Haj Ali. Tenset: A large-scale program performance dataset for learned tensor compilers. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*, 2021.